

Белорусский государственный университет
информатики и радиоэлектроники

Кафедра ЭВМ
Жизненный цикл разработки программного
обеспечения

Лабораторная работа №1

Выполнил: Левшук Д.Д.

Проверил: Внук О.М.

Минск 2026

1. Цель и задачи

Цель работы — изучить приёмы работы в интегрированной среде разработки (IDE) на примере Microsoft Visual Studio, получить практические навыки создания оконного приложения с графическим пользовательским интерфейсом на объектно-ориентированном языке программирования, а также изучить архитектурный шаблон MVC (Model — View — Controller) с активной и пассивной моделью.

Для достижения поставленной цели необходимо решить следующие задачи:

1. изучить состав и возможности интегрированной среды разработки, а также теоретические основы паттерна MVC;
2. установить и настроить выбранную среду разработки (Microsoft Visual Studio), ознакомиться с её средствами отладки;
3. разработать оконное приложение согласно варианту № 11 задания — программу для определения гликемического индекса продуктов;
4. реализовать в приложении архитектуру MVC с активной моделью;
5. продемонстрировать работу приложения и отладчика преподавателю, оформить отчёт по лабораторной работе.

2. Краткие теоретические сведения

2.1. Интегрированная среда разработки

Интегрированная среда разработки (Integrated Development Environment, IDE) — это комплекс программных средств, объединяющий в одном приложении инструменты, необходимые для полного цикла разработки программного обеспечения: текстовый редактор с подсветкой синтаксиса и автодополнением кода, компилятор или интерпретатор, систему сборки проекта, отладчик, менеджер проектов и файлов, а также, для приложений с графическим интерфейсом, визуальный конструктор форм.

В рамках данной работы использовалась среда Microsoft Visual Studio, которая для платформы .NET и языка C++/CLI предоставляет визуальный дизайнер форм Windows Forms, встроенный отладчик с возможностью установки точек останова, пошагового выполнения кода (Step Into/Over/Out) и просмотра значений переменных в окнах Watch и Locals, а также систему контроля версий и средства статического анализа кода.

2.2. Архитектурный шаблон MVC

MVC (Model — View — Controller) — архитектурный шаблон, разделяющий приложение на три взаимосвязанных компонента:

- Model (модель) — хранит данные приложения и реализует бизнес-логику: правила обработки и вычисления, не зависящие от способа отображения;

- View (представление) — отвечает за отображение данных пользователю и получение от него событий ввода (нажатия кнопок, ввод текста);

- Controller (контроллер) — принимает события от представления, проверяет (валидирует) входные данные и обновляет модель, а также, при необходимости, инициирует обновление представления.

Такое разделение снижает связанность компонентов и упрощает сопровождение и тестирование приложения: модель можно изменять независимо от интерфейса, а интерфейс — независимо от бизнес-логики.

2.3. Активная и пассивная модель

Различие между активной и пассивной моделью заключается в том, каким образом представление узнаёт об изменении данных.

При активной модели модель самостоятельно оповещает подписанные на неё представления об изменении своего состояния — как правило, с помощью механизма событий (event/delegate). Контроллер лишь передаёт модели новые данные, а модель сама инициирует обновление всех заинтересованных представлений. Такой подход удобен, если на одни и те же данные может быть подписано несколько представлений.

При пассивной модели модель не оповещает никого об изменениях: после каждого действия контроллер самостоятельно запрашивает у модели актуальное состояние и обновляет представление. В упрощённом варианте паттерна класс контроллера и пассивной модели совмещают в одном классе.

3. Описание выполненного варианта задания

3.1. Постановка задачи

Согласно варианту № 11 требуется разработать приложение для определения гликемического индекса продуктов. На главной форме приложения размещаются определение понятия «гликемический индекс» и кнопка «Ввести продукт». По нажатию на кнопку открывается

дополнительная форма, содержащая текстовое поле для ввода наименования продукта. Последнее введенное значение должно запоминаться главной формой и восстанавливаться в поле ввода при повторном открытии дополнительной формы. После подтверждения ввода на главной форме отображается результат — значение гликемического индекса введенного продукта. При вводе некорректных данных (пустое поле, продукт отсутствует в базе значений и т. п.) пользователю выводится сообщение об ошибке.

3.2. Архитектура приложения

Приложение построено по шаблону MVC с активной моделью и включает следующие основные компоненты:

- View — главная форма (MainForm), отображающая определение гликемического индекса, кнопку «Ввести продукт» и результат расчёта, а также дополнительная форма ввода продукта с текстовым полем и кнопками подтверждения/отмены;

- Model — класс, хранящий текущее введенное название продукта и соответствующее ему значение гликемического индекса (справочник «продукт — индекс»); при изменении данных модель генерирует событие, на которое подписана главная форма;

- Controller — обрабатывает нажатие кнопок формы ввода, выполняет проверку корректности введенных данных и, в случае успеха, передает данные в модель; при обнаружении некорректных данных инициирует показ окна с сообщением об ошибке.

Благодаря активной модели главная форма подписывается на событие изменения данных и автоматически обновляет отображаемый результат, не опрашивая модель самостоятельно после каждого действия пользователя.

3.3. Точка входа в приложение

Файл App.cpp содержит точку входа программы — функцию main, помеченную атрибутом [STAThread], что обязательно для приложений Windows Forms. В функции последовательно вызываются методы Application::EnableVisualStyles() и Application::SetCompatibleTextRenderingDefault(false) для включения визуальных стилей интерфейса, создается экземпляр главной формы GlycemicApp::MainForm, после чего вызовом Application::Run(%form) запускается цикл обработки сообщений Windows Forms.

4. Результат работы

В результате выполнения лабораторной работы разработано оконное приложение, соответствующее варианту № 11 задания. Ниже приводятся снимки экрана, иллюстрирующие работу приложения.

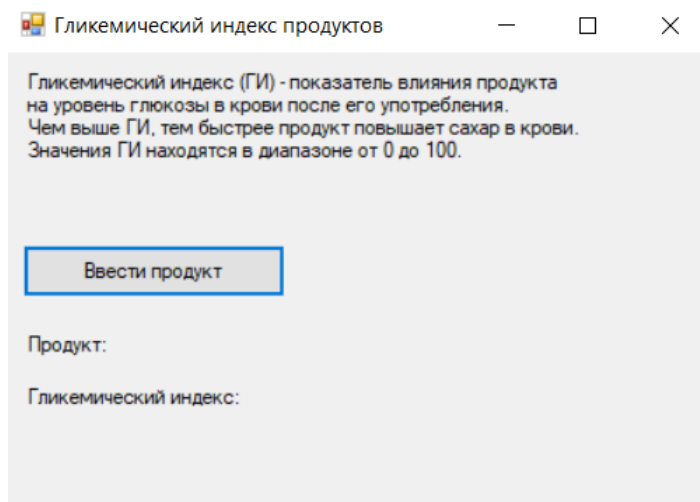


Рисунок 1 — Главная форма приложения

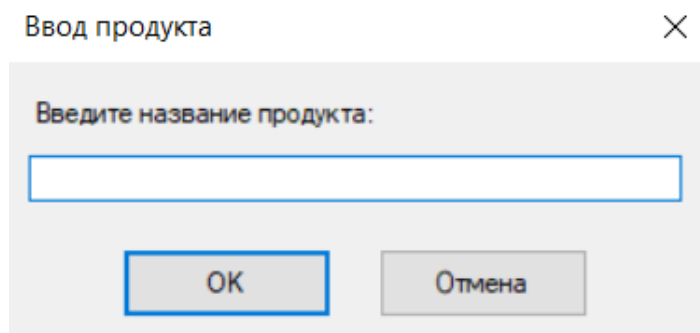


Рисунок 2 — Форма ввода продукта

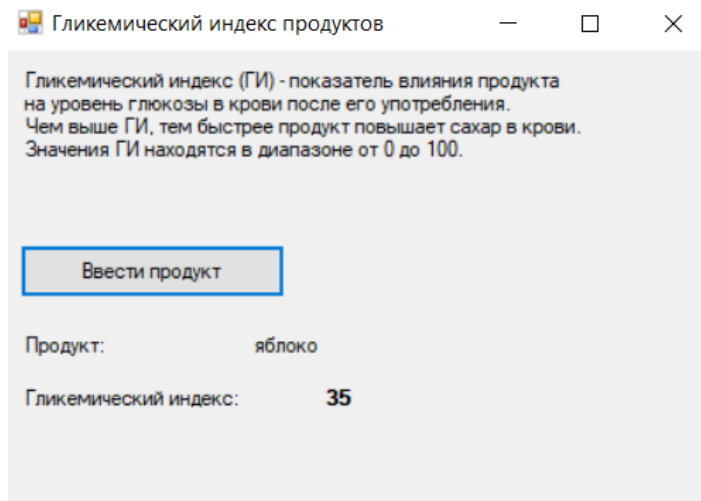


Рисунок 3 — Результат расчёта гликемического индекса на главной форме

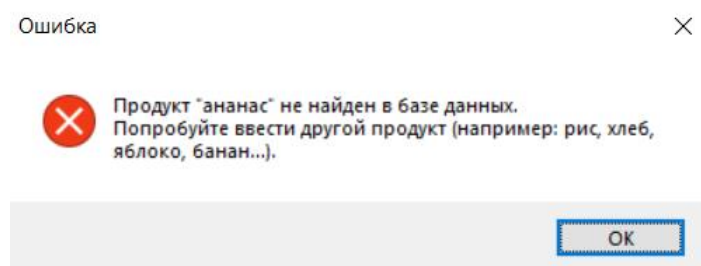


Рисунок 4 — Сообщение об ошибке при вводе некорректных данных

Приложение сохраняет последнее введенное значение и восстанавливает его при повторном открытии формы ввода, а также корректно обрабатывает некорректные входные данные, выводя соответствующее сообщение об ошибке. В ходе демонстрации преподавателю также была показана работа отладчика Visual Studio: установка точек останова в коде контроллера и модели, пошаговое выполнение (Step Into/Over) и просмотр текущих значений переменных в окне Watch.

Таким образом, задачи, поставленные в разделе 1, выполнены в полном объеме: изучены возможности IDE, разработано приложение с графическим интерфейсом, реализован каркас MVC с активной моделью.